Wiring and user guide: Arduino Uno + trainer + app

Versión en español: GUIA-USO.md.

This guide explains how to connect the trainer to a PC **using only an Arduino Uno** (no FTDI module), how to set it up the first time, and what to do when something fails. The app is in Spanish; button names are given as they appear on screen, with their meaning.

The Uno does two jobs over the same wires:

1. **ISP programmer**: writes the trainer's firmware through pins D10–D13.
2. **Serial bridge**: carries data between the PC app and the trainer through pins D2–D3.

There is no rewiring between the two jobs. The app picks the mode by itself.

```
PC (app) ──USB──> Arduino Uno ──D10..D13 (ISP)──> trainer's ATmega328P
                  (puente-uno) ──D2/D3 (serial)──>
```

1. What you need

- An Arduino Uno and its USB cable.
- 7 dupont wires (8 if the trainer is powered from the Uno).
- The trainer with its ATmega328P and 16 MHz crystal.
- The app: `app/build/bin/entrenador-bloques` (see app/README.md to build it).
- On Linux, permission to use the serial port: your user must be in the `dialout` group (check with `groups`). If you have already flashed the chip with avrdude through the Uno, you have it.

2. Wiring

Wire table

#	Arduino Uno		Trainer ATmega328P	Purpose
1	**D10**	→	**pin 1** (RESET)	ISP: resets the chip while flashing
2	**D11**	→	**pin 17** (MOSI, PB3)	ISP: data to the chip
3	**D12**	←	**pin 18** (MISO, PB4)	ISP: data from the chip
4	**D13**	→	**pin 19** (SCK, PB5)	ISP: clock
5	**D3**	→	**pin 2** (RXD, PD0)	Serial: what the Uno sends, the trainer receives
6	**D2**	←	**pin 3** (TXD, PD1)	Serial: what the trainer sends, the Uno receives
7	**GND**	—	**pin 8** (GND) or any GND on the board	Common ground. **Nothing works without it**

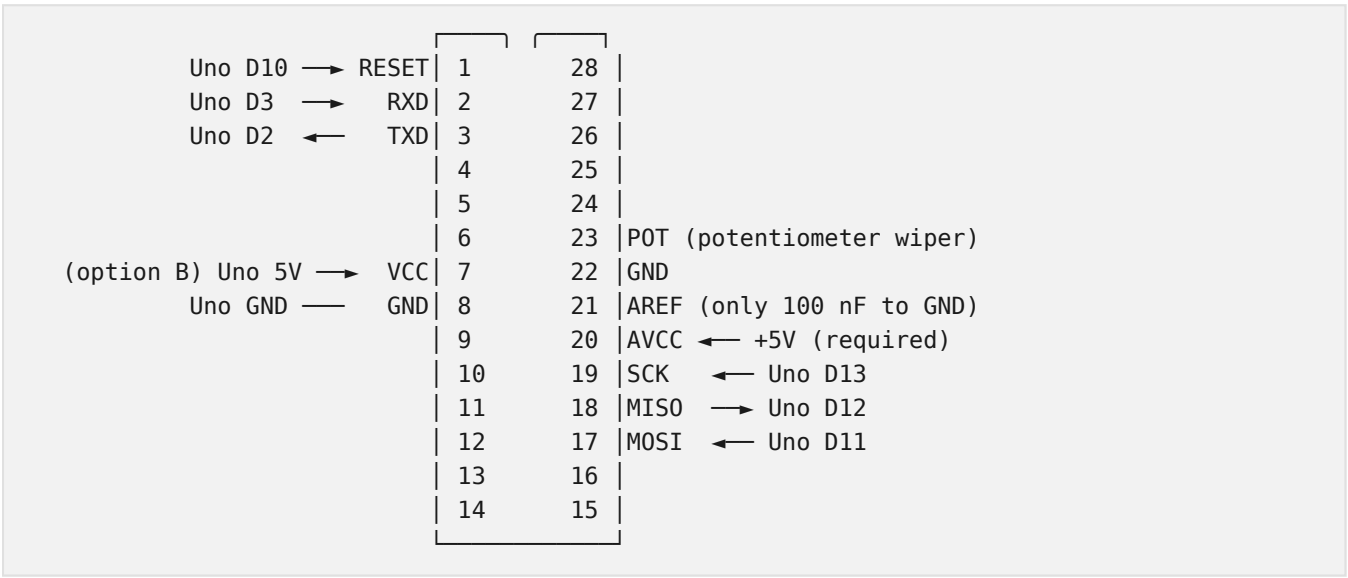
#	Arduino Uno		Trainer ATmega328P	Purpose
8	5V	→	pin 7 (VCC)	**Only** if the trainer has no power supply of its own (see below)

Serial is crossed: the transmitter (TX) on one side goes to the receiver (RX) on the other. Uno D3 transmits and goes to chip pin 2 (RX). Chip pin 3 (TX) goes to Uno D2.

> **Do not use Uno pins 0 and 1.** They are the Uno's own USB link. The bridge uses D2 and D3.

Where each ATmega328P pin is

Seen from above, with the chip's notch at the top (pin 1 has a dot next to it):



If your board has the connectors from the schematic, you can use them instead of the chip pins:

- **J3 (ICSP, 2×3)**: 1 MISO ← D12, 3 SCK ← D13, 4 MOSI ← D11, 5 RESET ← D10, 6 GND.
- **J2 (6 pins)**: 1 GND, 4 RXI ← D3, 5 TXO → D2. Do not connect pin 3 (VCC) to the Uno.

Power: pick one option only

- **Option A (recommended): the trainer has its own supply** (9–12 V jack or terminal block). Do not connect the Uno's 5V. Only GND is shared.
- **Option B: the trainer is powered by the Uno.** With nothing on the jack or terminal block, run a wire from the **Uno's 5V to pin 7 (VCC)** in addition to GND. Fine for testing, but USB gives little current for the displays.

Never both at once.

No capacitor on the Uno's RESET

Some ArduinoISP guides ask for a 10 µF capacitor between RESET and GND **on the Uno**. **Leave it out with the bridge.** The app resets the Uno when it opens the port to choose the mode; with the capacitor the Uno does not reset and the second connection fails.

Check before plugging in

- [ ] Multimeter on continuity: Uno GND ↔ ATmega pin 8 **beeps**.
- [ ] D2 goes to pin **3** and D3 to pin **2** (not the other way round).
- [ ] No wire on Uno pins 0 or 1.
- [ ] A single power source (option A or option B).
- [ ] The chip sits in its socket with the notch on the correct side.
- [ ] Pin 20 (AVCC) to +5V and pin 22 to GND: with the board powered, pins 20–22 read 5 V.
- [ ] The potentiometer's wiper (middle leg) goes to pin **23**, not 27 (that is DIP 4).

3. One-time setup

Do this the first time, or after changing the Uno or the trainer's chip.

3.1 Open the app and pick the port

1. Plug the Uno in over USB.
2. Open the app:

   ```
   cd entrenador-atmega328p/app
   ./build/bin/entrenador-bloques
   ```

3. Top right, in the port list, choose the Uno: on Linux, `/dev/ttyACM0 — Arduino Uno` (it can come back as `ttyACM1` after a replug). If it is missing, click ⟳.

3.2 Install the bridge on the Uno

1. Click **Firmware** (top right).
2. Click **Grabar puente en el Uno** (*flash bridge to the Uno*). It takes about 5 seconds.
3. It must end in green. If avrdude says `not in sync`, check that the Uno has no capacitor on RESET and try again.

This replaces ArduinoISP on the Uno. The bridge **is still ArduinoISP** ("Arduino as ISP" in the Arduino IDE works the same) and can also act as a serial bridge.

3.3 Flash the trainer's firmware

1. In the same window, click **Grabar firmware en el entrenador** (*flash firmware to the trainer*). It takes 20 to 40 seconds.
2. The avrdude output must show `Device signature = 1E 95 0F` and end in green.
3. Close the Firmware window.

This writes the `bloques` firmware, the 16 MHz fuses (L 0xFF, H 0xDE, E 0xFD) and the bootloader. **It erases the program saved** on the trainer.

4. Everyday use

1. Plug the Uno in over USB (and the trainer's supply, for option A).
2. Open the app, pick the port and click **Conectar** (*connect*). It takes about 3 seconds.
3. **Puente** (*bridge*) and **Entrenador** (*trainer*) must turn green. The log (bottom right) shows:

```
Abriendo /dev/ttyACM0...
Puente activo
Entrenador conectado
```

4. The **Entrenador** panel updates by itself: turning the potentiometer moves the blue bar and flipping the DIP switches moves the switches. Click an LED to light it on the board.
5. Write the program in **Diagrama** (*diagram*) or **Código** (*code*), or pick one from **Ejemplos...** (*examples*). **</> Generado** shows the same program as an Arduino sketch and as the bytecode the trainer receives.
6. ▶ **Ejecutar** (*run*, or Ctrl+Enter in the code editor) sends it to the trainer and it starts at once. ■ **Detener** (*stop*) stops it.
7. **Guardar en la placa** (*save to board*) keeps it in the trainer's memory: it runs by itself at power-up, even without the Uno or the PC. **Borrar de la placa** (*erase from board*) removes it.
8. When you are done, **Desconectar** (*disconnect*) or close the app.

No firmware reflash is needed to change programs: **Ejecutar** is enough.

5. Manual test without the app

Tells you whether a problem is in the wiring or in the app. With the app closed:

```
picocom -b 19200 --omap crlf --echo /dev/ttyACM0
```

Wait 2 seconds (the Uno resets when the port opens) and type:

Type	Expected reply	What it tests
`!`	`PUENTE OK`	The Uno has the bridge
`?` and Enter	`OK ENTRENADOR 1`	Wires D2/D3/GND and the trainer's firmware
`T` and Enter	`T 512 0 0 - 0 0` (pot, DIP, LEDs, number, running, error)	Inputs: the first number changes as you turn the potentiometer
`L 255` and Enter	`OK`, and all 8 LEDs light up	Outputs
`N 42` and Enter	`OK`, and the displays show 42	Displays

Leave picocom with **Ctrl+A** then **Ctrl+X**.

6. Troubleshooting

Symptom	Likely cause	What to do
The port is not in the list	Uno unplugged, or a charge-only USB cable	Try another cable or USB port; click ⟳
`permission denied` when opening the port	User not in `dialout`	Add it (on NixOS: `extraGroups = [ "dialout" ]` in the configuration) and log in again

Symptom	Likely cause	What to do
`el Uno no respondió como puente` (*the Uno did not answer as a bridge*)	The Uno still has plain ArduinoISP	Do step 3.2
Same, but only the second time	10 µF capacitor on the Uno's RESET	Remove it
Same, with another program open	The Arduino IDE or picocom holds the port	Close the serial monitor
Puente green, **Entrenador** red	D2 and D3 swapped	Swap them: D3 → pin 2, D2 ← pin 3
Same, with D2/D3 correct	No common GND, trainer unpowered, or no `bloques` firmware	Check GND with the multimeter, the supply, and do step 3.3
avrdude: `Device signature = 00 00 00`	No common GND, trainer unpowered, or a loose ISP wire	Check GND and wires 1–4 of the table
avrdude: `Device signature = FF FF FF`	MISO/MOSI crossed, or the crystal does not oscillate	D11 → pin 17, D12 → pin 18; check the crystal and its 22 pF capacitors
avrdude: `not in sync` / `protocol error`	The Uno has neither the bridge nor ArduinoISP	Do step 3.2
Connects, but the monitor does not change	The running program overwrites the LEDs	Click ■ **Detener** and try by hand
The potentiometer moves by itself, takes seconds to settle, or never reaches 0 and 1023	AVCC (pin 20) not on +5V, or the wiper on the wrong pin	Measure between pins 20 and 22: it must read 5 V. The wiper goes to pin 23. Wired correctly, it varies ±2 when still
Wrong segments light up	Different bus wiring	Check with `firmware/prueba-mapeo` and the `BIT_SEG` table

7. How it works inside

- The Uno runs `firmware/puente-uno`. When the port opens it resets and waits:
  - If avrdude commands arrive, it works as ArduinoISP.
  - If the character `!` arrives, it replies `PUENTE OK` and from then on forwards everything between USB and D2/D3 (SoftwareSerial at 19200 baud), until the next reset.
- The trainer runs `firmware/bloques`. It receives the program compiled by the app and runs it in a virtual machine. The text command protocol (`?`, `T`, `L`, `N`, `D`, `E`, `R`, `S`, `W`, `X`) is described at the top of that file.
- The hardware layer is the `Entrenador` Arduino library, shared by the firmware and by the sketches the app generates.
- The app is `app/`; the language and the diagram live in `app/lenguaje/`.